

REASONING MODELS

From o1 to R1 — How RL Made LLMs Think

RLVR + GRPO + Emergent Self-Correction

The Day Everything Changed

September 12, 2024

OpenAI releases o1-preview

Same week:

GPT-4o

13% on AIME math

o1-preview

56% on AIME math

o1

83% on AIME math

Same architecture. Same scale. Different training.

What Was Different About o1?

Old way:

Prompt → instant answer

o1's way:

Prompt → [thinks 30s] → answer

OpenAI's blog:

"We trained these models to spend more time thinking through problems before they respond, much like a person would."

Thinking time isn't padding — it's the model exploring, backtracking, double-checking.

System 1 vs System 2 (Kahneman)

System 1:

fast, intuitive, automatic

"What's $2+2$?" → 4 (instant)

standard LLM behavior

System 2:

slow, deliberate, analytical

"What's 17×23 ?" → pause, calculate, 391

o1's mode

Pre-o1 LLMs were ALL System 1.

o1 added System 2 — for the first time.

The Mystery

OpenAI told us what o1 could do.

They did NOT tell us how they trained it.

What we knew:

- ▶ Used reinforcement learning
- ▶ Generated long chains of thought
- ▶ CoT was hidden ("safety + competitive advantage")

What we DIDN'T know:

- ▶ The algorithm
- ▶ The reward function
- ▶ The training data
- ▶ Whether SFT was needed first
- ▶ Whether reasoning was learned or imitated

For 4 months, the community had a working reasoning model and zero understanding of how.

Then — January 20, 2025

DeepSeek releases R1

Same reasoning capability as o1.

Open weights.

Open paper.

With the recipe.

The world finally got to see
how reasoning models are made.

Why R1 Matters Beyond Capability

o1 said:

"Reasoning models exist."

R1 said:

"Here's how to build one."

Three breakthroughs in the R1 paper:

1. Pure RL works (no SFT needed for R1-Zero)

2. Reasoning EMERGES (was not taught)

3. The algorithm: GRPO (simpler than PPO)

How LLMs Got Better Until Now

The old scaling laws (Kaplan, Hoffmann):

More parameters → better

More training data → better

More training compute → better

Better == lower loss, better benchmark scores

For 5 years, "make it bigger" was the recipe.

The New Scaling Axis

o1 introduced a fourth axis:

More TEST-TIME compute → better

i.e., let the model think longer at inference time — better answer.

- ▶ "Thinking" tokens cost real money and time.
- ▶ But they pay for themselves in accuracy.

For the first time, scaling continues at inference, not just training.

The Three Scaling Curves

Performance gains come from:

1. Model size (parameters) ← pre-training era

2. Training compute (FLOPs) ← pre-training era

3. Test-time compute (think tokens) ← NEW

OpenAI showed all three curves in the o1 launch. Test-time scaling was as steep as the others.

Implication:

Even small models can be made smart
by giving them more thinking time.

The Tradeoff

Test-time compute is NOT free:

Pros:

- ▶ Better answers on hard problems
- ▶ Smaller models can match bigger ones
- ▶ Self-correction catches errors

Cons:

- ▶ 10-100x more tokens per response
- ▶ 10-100x slower
- ▶ 10-100x more expensive
- ▶ Latency hurts UX

Use reasoning models only when reasoning matters.

PART 2

THE DEEPSEEK R1 STORY

"This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning."

— DeepSeek R1 paper

R1-Zero — The Pure RL Experiment

The hypothesis:

Can a base model learn to reason with NO supervised fine-tuning?

Standard pipeline:

Pre-train → SFT → RLHF/DPO

DeepSeek's R1-Zero pipeline:

Pre-train → SKIP SFT → straight to RL

↑ This was supposed to be impossible

Recall from 10a: you MUST start RL from an SFT'd model.

R1-Zero violated that rule deliberately.

The Setup

Base model: DeepSeek-V3-Base (671B MoE)

Algorithm: GRPO (their own RL algorithm)

Reward: ONLY answer correctness + format

No reward model. No human labels. No SFT.

Just: "did you get the math problem right?"

Tasks:

- ▶ Math (AIME, MATH-500)
- ▶ Coding (LiveCodeBench, Codeforces)
- ▶ Logic (formal reasoning)

Reward = 1 if correct, 0 if wrong. That's it.

What Happened

AIME 2024 pass@1:

Step 0	(base model):	15.6%
Step 1000:		30%
Step 5000:		55%
Step 8000:		71%
Final:		77.9%

For comparison:

GPT-4o:	13%
o1-0912:	74%

Pure RL on math correctness —
the model matches o1 on AIME.

The "Aha Moment"

Around training step 8000, something strange:

The model started writing things like:

"Wait, let me reconsider..."

"Actually, that's not right..."

"Let me try a different approach..."

**These behaviors WERE NOT IN THE TRAINING DATA.
The reward function never said "write 'wait'."**

**The model learned to backtrack, reflect, and self-correct —
purely because doing so led to correct final answers.**

What the Aha Moment Means

RL doesn't just polish behaviors.
It can create NEW behaviors not in the data.

Reasoning $\neq$ memorization of reasoning patterns.
Reasoning IS reward-seeking search.

You don't need to train the model to "think."
You need a reward that rewards correctness.
Thinking emerges as the means to the end.

R1-Zero, any problems?

Strengths:

- ▶ Strong reasoning
- ▶ Emergent self-correction

The Full R1 Recipe (4 stages)

Stage 1

Cold-start SFT (small)

Few thousand high-quality reasoning examples — a "head start"

Stage 2

RL on reasoning (GRPO)

Like R1-Zero, but starting from the SFT'd model. More readable.

Stage 3

Rejection sampling + SFT

Generate, filter for quality, retrain. Adds general task capability.

Stage 4

Final RL with RLHF + RLVR

Combine human preference + verifiable rewards.

Result: R1 — readable AND reasoning-capable

The Two Lessons from R1

Lesson 1: Pure RL works (R1-Zero proved it)

- ▶ Reasoning is not just memorization
- ▶ Emergence is real

Lesson 2: But pure RL alone isn't enough (R1 showed it)

- ▶ For production, mix RL + SFT
- ▶ Use SFT for format, RL for capability

The breakthrough was DEMONSTRATING #1.

The practical recipe is #2.

PART 3

RLVR

Reinforcement Learning with Verifiable Rewards

The paradigm behind it all.

The Reward Problem We Saw in 10a

RLHF rewards come from a model:

```
Reward Model (trained on human preferences)
```

↓

```
Score the response
```

Problems:

- ▶ Reward model is approximate
- ▶ Reward hacking is rampant
- ▶ Training the reward model takes effort
- ▶ Rewards are subjective

Goodhart's Law: optimize a proxy, the proxy gets corrupted.

The RLVR Insight

For SOME tasks, we don't need a model to score.

We can write a PROGRAM.

```
Math:    if int(answer) == expected:  reward = 1
Code:    if all_tests_pass(code):      reward = 1
JSON:    if parses_validly(output):    reward = 1
Logic:   if proof_checker(out):        reward = 1
```

Verifiable rewards = programmatic checks

= deterministic, cheap, perfect

RLHF vs RLVR Side-by-Side

	RLHF	RLVR
Reward source:	Reward model	Program/verifier
Reward signal:	Continuous	Binary (or stepped)
Reward training:	Needs data	Just write code
Reward hacking:	Common	Rare (verifier is exact)
Domain:	Subjective tasks	Objective tasks

You can do BOTH on the same model.

(R1 used both in stage 4)
Different rewards for different tasks.

When RLVR Works (And When It Doesn't)

WORKS:

- ▶ Math (compare numerical answers)
- ▶ Code (run tests, count passes)
- ▶ Logic (formal proof checker)
- ▶ Format compliance (regex)
- ▶ JSON validity (parser)
- ▶ Game playing (game state)

DOESN'T WORK:

- ▶ Creative writing (no "correct")
- ▶ Helpfulness (subjective)
- ▶ Translation (multiple valid)
- ▶ Conversation (no ground truth)
- ▶ Style (preference based)

**The split is roughly: objective vs subjective.
If you can write the verifier, RLVR works.**

The R1 Reward Function (Concrete)

```
def reward(response, ground_truth):  
    score = 0  
  
    # Format reward – encourage <think>...</think>  
    if "<think>" in response and "</think>" in response:  
        score += 0.1  
  
    # Correctness reward – does final answer match?  
    final = extract_answer(response)  
    if final == ground_truth:  
        score += 1.0  
  
    return score
```

That's it. No reward model. No labels. Just rules.

Why Format Reward Matters

Without format reward:

- ▶ Correct answers in random formats
- ▶ Hard to extract for evaluation
- ▶ Hard to use downstream

With format reward:

- ▶ Wraps reasoning in <think> tags
- ▶ Reasoning is parseable and visible
- ▶ Model can be evaluated reliably
- ▶ Chain of thought becomes structured

Format reward is small (0.1) so it's a NUDGE,
not the main signal. Correctness is the main signal.

Format helps the engineering. Correctness drives the capability.
Get the ratio wrong and you get format-only optimization without learning.

The RLVR Limitation

RLVR can ONLY improve what the base model can do.

If the base model never produces the correct answer,
RL can never reward it, and never reinforce it.

Recent research (2025):

**RLVR mostly improves SAMPLING EFFICIENCY,
not raw capability.**

i.e., RL teaches the model to FIND its existing
knowledge faster, not to learn new knowledge.

This is a hot debate. The capability/efficiency question is unresolved.

PART 4

GRPO

Group Relative Policy Optimization

The algorithm that powers DeepSeek R1

PPO Recap (from 10a)

PPO objective (with all the machinery):

$$L = E[\min(r_t \cdot A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \cdot A_t)]$$

Where:

$$r_t = \pi_{\theta}(a|s) / \pi_{\text{old}}(a|s) \quad (\text{policy ratio})$$

$$A_t = \text{advantage} \quad (\text{how much better than expected})$$

To compute A_t we need a VALUE MODEL (critic):

$$V(s) = \text{"what reward should I expect from state } s\text{"}$$

$$A = \text{actual_reward} - V(s)$$

Critic = a separate neural net trained alongside the policy.

The Cost of the Value Model

PPO's hidden cost: the value model

For an LLM, the value model is also a neural net, roughly the same size as the policy.

- Memory:** doubles your training footprint
- Compute:** doubles your gradient computation
- Stability:** critic training is its own headache

For a 7B model:

Policy:	14 GB
Reference:	14 GB
Reward:	14 GB
Value:	14 GB
Total:	56+ GB just for weights

The GRPO Insight

What if we DON'T need a value model?

What if we estimate "how good is this response"

by COMPARING IT TO OTHER RESPONSES TO THE SAME PROMPT?

```
Generate 8 responses to the same question.
```

```
Score each:  $r_1, r_2, \dots, r_8$ 
```

```
Compute  $\text{mean}(r)$  and  $\text{std}(r)$ 
```

```
 $\text{Advantage}_i = (r_i - \text{mean}) / \text{std}$ 
```

```
"How much better than the AVERAGE response?"
```

No value model needed.

Why GRPO Works (Intuition)

Why is "above average" a useful signal?

- ▶ Same prompt → 8 different reasoning attempts
- ▶ Some lead to correct answers, some don't

REWARD

the ones above average

PUNISH

the ones below average

- policy shifts toward what works for THIS prompt
- no need to predict global reward distributions

The "group" in GRPO = the 8 generations from one prompt.

The "relative" in GRPO = each compared to its peers.

The GRPO Equation

```
L_GRPO = (1/G) Σ_i [  
    min( r_i · A_i, clip(r_i, 1-ε, 1+ε) · A_i )  
    - β · KL(π_θ || π_ref)  
]
```

Where:

G = group size (number of generations, typically 8)

r_i = $\pi_\theta(o_i|q) / \pi_{\theta_{old}}(o_i|q)$ policy ratio

A_i = $(R_i - \text{mean}(R)) / \text{std}(R)$ group-normalized advantage

KL = anchor to reference model

Compare to PPO: Same min/clip. Same KL.

DIFFERENT advantage: empirical mean vs critic.

GRPO Step-by-Step

For each training prompt q :

1. Generate G responses: $o_1, o_2, \dots, o_G$ (using current policy)

2. Score each with the verifier: $R_1, R_2, \dots, R_G$

3. Normalize within group: $A_i = (R_i - \text{mean}) / \text{std}$

4. Update policy weights to: $\uparrow$ high- A , $\downarrow$ low- A

5. Apply KL penalty to stay near reference

6. Repeat with next prompt

Group Size Matters

G too small (G=2):

- ▶ Advantage is noisy
- ▶ Hard to distinguish good from average

G too large (G=32):

- ▶ Compute scales linearly
- ▶ Diminishing returns past ~8

Sweet spot: G = 8 (used in R1)

Small models with limited compute: G = 4 (workable)

GRPO vs PPO — Resource Comparison

	PPO	GRPO
Models in memory:	4	3 ← no value model!
Training stability:	Tricky	Stable
Hyperparameters:	12+	5
Compute per step:	High	Lower
Generation:	Sequential	Parallel (group)

Compatibility:

PPO works for any reward (including model-based)

GRPO works best when reward is cheap to compute

**RLVR + GRPO is a perfect match —
verifiable rewards ARE cheap, so we can afford G=8 generations.**

A Subtle Point — The KL Penalty

GRPO keeps a KL penalty to a reference model:

$$- \beta \cdot \text{KL}(\pi_{\theta} || \pi_{\text{ref}})$$

Why? Same reason as PPO/RLHF/DPO: without anchor, the model exploits the verifier.

Without KL:

Model writes "<think>I'll cheat by..."
high format score, low correctness, but pattern works
→ drift, instability, gibberish

With KL:

Stays close to base behavior, reasoning emerges organically from the original linguistic capabilities.

Same role. Same intuition. β typically 0.001-0.01.

Why GRPO + RLVR Matched Up Perfectly

GRPO needs:

Cheap reward signal
(since we score G=8 per prompt)

→ can't afford a slow learned reward model

RLVR provides:

Programmatic verifier
(microseconds per check)

→ exactly what GRPO wants

Designed for each other.

PPO + RLHF was the old pair.

GRPO + RLVR is the new pair.

The pair RLVR + GRPO is the foundation of every post-2025 reasoning model.

Today's Hands-On — A Controlled Failure

Train SmolLM-135M with GRPO on GSM8K math.

This is going to fail. ON PURPOSE.

Why on purpose?

Unsloth's docs explicitly warn:

```
"Apply GRPO to a model at least 1.5B in parameters  
to correctly generate thinking tokens"
```

SmolLM-135M is 11× smaller than that floor.

What we'll observe:

- ▶ The algorithm runs perfectly
- ▶ The reward functions fire perfectly
- ▶ The model learns NOTHING

The failure IS the lesson.

The 5 Reward Functions Stack

Will Brown's GSM8K reward stack (from Unsloth):

Function	Max Reward	What it measures
correctness_reward	+2.0	Final answer correct?
int_reward	+0.5	Answer is an integer?
strict_format_reward	+0.5	Exact XML structure?
soft_format_reward	+0.5	XML structure (lenient)?
xmlcount_reward	+0.5	Partial credit per tag
Total max:	+4.0	

The graduated rewards (xmlcount in particular) are designed so the model gets SOME signal even before it produces full correct answers.

Watch xmlcount carefully — it's the canary in the coal mine.

The Diagnostic — Three Signals

Three numbers tell you whether GRPO is working:

Signal 1

`correctness_reward > 0`

"Is the model getting math right?"

Signal 2

`reward_std > 0`

"Are the 4 generations producing different rewards?"

"If all 4 get 0 reward, NO learning signal."

Signal 3

`xmlcount_reward > 0`

"Is the model producing ANY structure?"

"If 0, the algorithm has nothing to amplify."

ALL THREE must be > 0 for GRPO to learn.

If any are 0, the run is dead.

What We Saw — Steps 1 through 18

The actual training table from our run:

Step	reward	reward_std	correctness	xmlcount	kl
1	0.00	0.00	0.00	0.00	0.00
2	0.00	0.00	0.00	0.00	0.00
3	0.00	0.00	0.00	0.00	0.00
...					
18	0.00	0.00	0.00	0.00	0.00

Every column. Every row. Exactly zero.

- ▶ Model generates 60-512 token completions
- ▶ None contain a <reasoning> tag
- ▶ None get math right
- ▶ All 4 generations per prompt get identical (zero) reward

This is the failure mode, in numbers.

The Zero Cascade

xmlcount = 0 (model produces no XML tags)

soft_format = 0 (no <reasoning>...<answer> structure)

strict_format = 0 (no exact format match)

int_reward = 0 (extracted answer isn't even an integer)

correctness = 0 (extracted answer never matches truth)

ALL rewards = 0 (reward_std = 0, advantage = 0)

NO gradient (KL = 0, weights frozen)

The cascade ALL starts at the top:

the model can't even produce the format.

The Lesson From Our Run

What we just demonstrated:

RL polishes existing behaviors.

RL amplifies existing patterns.

RL does NOT teach behaviors from scratch.

For RL to work:

The base model must occasionally produce the target behavior — even by accident.

Then RL increases the frequency.

If the base model can't even produce the target

ONCE in 100 generations, RL cannot help.

When To Reach For RL (And When Not To)

You want to add capability X to your model.

Is X already in the base model AT ALL?

NO → Use SFT first

(teach the pattern from imitation)

YES, rarely → Use RL (RLHF, DPO, GRPO)

(amplify the existing rare behavior)

YES, often → Maybe RL for refinement,

maybe just better prompting

The R1 recipe sequences these correctly:

Pre-train → Cold-start SFT → RL (GRPO)

"build the capacity" "teach the format" "amplify the capability"

Reward Hacking Even in RLVR

RLVR is NOT immune to reward hacking:

- ▶ Model finds tricks that pass tests but aren't right
- ▶ Model produces verbose <think> tags with empty content
- ▶ Model exploits regex format reward without reasoning

Defense:

- ▶ Multiple reward functions (correctness + format + length)
- ▶ Adversarial test sets (verifier coverage)
- ▶ KL penalty to reference

Goodhart's Law is undefeated.

Even with deterministic rewards.

The Limitations

1. Only works for tasks with verifiable answers
2. Capability ceiling = base model's reasoning paths
3. Reward hacking still happens
4. Long chains = expensive at inference
5. Hidden CoT raises safety concerns (alignment debate)
6. Unclear if reasoning generalizes outside training domain

Reasoning models are not omniscient.

They are math/code/logic specialists
with emergent self-correction.

The 10a → 10b Cheat Sheet

Need:

Use:

Domain knowledge

→ **CPT (9a)**

Format/instructions

→ **SFT (9b)**

Subjective quality

→ **RLHF / DPO (10a)**

Verifiable correctness

→ **RLVR + GRPO (10b)**

Production reasoning model:

CPT → SFT → DPO → RLVR

Each step adds something the previous step couldn't.

Which Should I Use?

Build a math tutor

→ **RLVR**

(math is verifiable)

Build a customer support bot

→ **RLHF** / **DPO**

(helpfulness is subjective)

Build a code generator

→ **RLVR**

(tests are verifiable)

Build a creative writing assistant

→ **DPO**

(style is preference-based)

Build a research assistant

→ **BOTH**

(RLHF for help, RLVR for facts)

Match the reward type to the task.

The Future After R1

Already happening (2025-2026):

- ▶ Multi-turn RLVR (agent + environment loops)
- ▶ Process rewards (intermediate step verification)
- ▶ Self-improvement (AI-generated training problems)
- ▶ Tool use within reasoning (Code Interpreter, search)
- ▶ Reasoning distillation (small model, big mind)

What we don't know yet:

- ▶ Does reasoning generalize beyond math/code?
- ▶ Can RLVR scale to truly novel domains?
- ▶ What's the limit of test-time compute scaling?

Final Takeaways

1. Reasoning emerged from RL — not designed in (R1)
2. Test-time compute is the new scaling axis
3. RLVR replaces reward models with verifiers
4. GRPO replaces value models with group statistics
5. Pure RL works (R1-Zero) — IF the base model is large enough
6. The model size floor for GRPO is ~1.5B parameters
7. RL amplifies. RL polishes. RL does NOT teach from scratch.

The algorithm we ran today was identical to R1's.

The only thing different was the base model.

Capacity is the floor. Algorithms work above it, not below it.